

第 04 章 文件系统层次、路径、文件类型与链接

从名称到对象：沿路径分量看清目录项、inode、链接与删除后的生命周期。

对象模型操作语义验证诊断经典任务

大字号阅读版

本章阅读导航

先抓住一条主线：路径只是逐级查找名称的指令；目录项把名称映射到 inode；真正的对象身份、链接行为和删除结果必须从 inode 与剩余引用中判断。

- 01 目录树与起点先判断路径从 ``/`` 还是当前目录开始
- 02 逐级解析把路径拆成分量，找第一处无法继续的位置
- 03 名称与 inode 把目录项、inode 和对象数据分开
- 04 文件类型区分名称后缀、inode 类型和内容格式
- 05 硬链与软链判断同 inode 还是独立路径对象
- 06 验证与诊断按名称、链接原文、路径链和对象身份取证

专题地图

- 知识专题单根目录树与 FHS 职责
- 知识专题路径解析与逻辑/物理路径
- 知识专题目录项、inode 与对象生命周期
- 知识专题文件类型与证据边界
- 操作专题从路径表面深入真实对象
- 知识专题硬链接与符号链接
- 操作专题创建、验证、替换和删除链接
- 诊断专题逐级定位路径和链接故障
- 经典任务修复相对链接；观察删除后的分化

阅读时持续回答

1. 这条路径从哪里开始解析？
2. 第一个失败的路径分量是哪一个？
3. 我看到的是名称、链接文本还是最终对象？
4. 两个名称是否处于同一文件系统并共享 inode？
5. 当前命令会观察链接自身还是跟随目标？
6. 删除的是哪个目录项，还有哪些引用存在？
7. 这条证据能证明什么，又不能证明什么？

第 04 章 · 正文

在命令行里看到 `/srv/app/current/config.yml` 时，人很容易把它当作一个完整、固定的“文件”。Linux 实际处理的是一串路径分量：从根目录或当前工作目录出发，在每一级目录中查找名称；名称通过目录项映射

到 inode；inode 再描述对象类型、元数据、链接计数以及数据位置。路径中还可能插入符号链接保存的目标文本，所以表面名称与最终对象可能完全不同。

如果把名称、对象和内容混在一起，就会产生高频误判：内容相同就认为是同一个文件；删除“原文件”就认为对象已经消失；`readlink` 有输出就认为目标可访问；硬链接失败后改用复制却仍声称“同 inode”；路径报错时只检查最终文件，而没有寻找第一个失败的目录分量。

本章以“路径文本如何解析到对象”为主线，先建立目录树、目录项、inode 和文件类型模型，再用 `pwd`、`stat`、`readlink`、`realpath` 与 `namei` 分层取证，最后解释硬链接、符号链接和删除名称后的生命周期。传统权限留给第 08 章，完整的 `find` 留给第 05 章，挂载遮蔽与持久配置留给第 24 章。

概念

目录项（**directory entry**）是目录中“名称到 inode”的映射。路径解析每前进一级，都是在当前目录中查找下一个目录项。名称属于目录项而不是普通文件 inode，因此重命名、硬链接和删除名称首先改变的是目录中的映射关系。

概念

inode 是文件系统内部描述对象身份与元数据的记录，包含对象类型、模式位、所有者、大小、时间、链接计数以及数据引用。inode 编号只在所属文件系统内唯一，所以比较对象身份至少要同时核对文件系统标识和 inode。

概念

路径解析（**path resolution**）是从某个起点逐级解释路径分量的过程。中间分量必须能够继续作为目录使用；遇到符号链接时，系统把链接保存的目标文本插入剩余路径继续解析。诊断的关键不是重复访问最终文件，而是找到第一处无法继续的分量。

概念

绝对路径与相对路径的差别首先是解析起点：绝对路径从 `/` 开始，相对路径从进程当前工作目录开始。`.` 与 `..` 是路径分量；波浪号 `~` 则属于 Shell 展开，不是内核路径解析规则。

概念

文件类型是 inode 的对象属性，普通文件、目录、符号链接、块设备、字符设备、FIFO 和 socket 的操作语义不同。扩展名只是名称的一部分，`file` 判断的是内容或格式；名称后缀、inode 类型和内容格式必须分开取证。

概念

硬链接（**hard link**）是另一个指向同一 inode 的目录项。多个硬链接是平等名称，不存在永久的“原件”和“副本”；它们必须位于同一文件系统，普通工作流也不为目录建立硬链接。删除一个名称只会减少链接计数。

概念

符号链接（**symbolic link**）是拥有自己 inode 的独立对象，其数据是一段目标路径文本。它可以跨文件系统、可以指向目录、也可以在目标不存在时创建；代价是目标移动或删除后可能悬空，并在同一路径后来出现新对象时重新绑定。

操作语义

以下入口构成本章最小接口地图。先明确命令观察或改变的对象，再选择参数；正文专题会继续解释证据边界，不在每个小节点重复整套手册页。

pwd / realpath

SYNOPSIS

```
pwd [-LP]
realpath [OPTION]... FILE...
```

确认相对路径的解析起点，并在需要时展开符号链接得到规范路径。

重要参数 / 形式

pwd

显示 Shell 记录的逻辑工作目录。

pwd -P

显示展开符号链接后的物理工作目录。

realpath -e PATH

要求所有分量存在，用于证明当前路径完整可解析。

realpath -m PATH

允许分量缺失，适合规范化计划创建或当前失效的路径文本。

ls / stat / file

SYNOPSIS

```
ls [OPTION]... [FILE]...
stat [OPTION]... FILE...
file [OPTION]... FILE...
```

分别观察目录项摘要、inode 元数据和内容格式；三者回答的是不同层次的问题。

重要参数 / 形式

ls -ld -- PATH

查看名称自身；**-d** 避免把目录展开为内容列表。

stat -- PATH

默认不解引用最终符号链接，因此可观察链接自身的 inode。

stat -L -- LINK

跟随最终符号链接，观察目标对象。

%d / %i / %h / %F / %N

设备标识、inode、硬链接计数、类型和名称/链接表示。

file -L -- LINK

跟随链接判断目标内容格式；它不证明两个名称共享 inode。

ln

SYNOPSIS

```
ln [OPTION]... TARGET LINK_NAME
```

默认创建硬链接：在目标目录中增加一个指向现有 inode 的新名称。

重要参数 / 形式

```
ln TARGET HARD_LINK
```

要求目标存在，并且新名称与目标位于同一文件系统。

```
--
```

结束选项解析，避免以 `-` 开头的名称被当作选项。

跨文件系统

会失败并出现类似 `Invalid cross-device link` 的错误；不能用复制后仍声称“同一 inode”。

目录硬链接

普通管理员工作流不使用；题目要求链接目录时选择符号链接。

```
ln -s
```

SYNOPSIS

```
ln -s [OPTION]... TARGET_TEXT LINK_NAME
```

创建独立的符号链接对象，并把 `TARGET_TEXT` 原样保存为链接数据。

重要参数 / 形式

```
-s
```

创建符号链接而不是硬链接。

```
-r
```

与 `-s` 配合，按链接位置生成相对目标；使用前仍要验证最终文本。

相对目标

从符号链接所在目录解释，而不是从创建命令时的当前目录解释。

```
-f
```

会删除已有目标名称，风险较高；本章不把无调查的 `ln -sf` 当作默认修复。

```
readlink
```

SYNOPSIS

```
readlink [OPTION]... FILE...
```

无参数时读取符号链接保存的原始目标；带规范化参数时可跟随路径链。

重要参数 / 形式

```
readlink -- LINK
```

只显示链接保存的原始文本，不保证目标存在。

```
readlink -f PATH
```

递归跟随符号链接并规范化；除最后一个分量外必须存在。

```
readlink -e PATH
```

要求所有路径分量都存在。

```
readlink -m PATH
```

允许分量缺失；结果不是“当前对象存在”的证明。

namei

SYNOPSIS

```
namei [OPTION]... PATHNAME...
```

沿路径逐级显示分量，遇到符号链接时展开目标，是定位第一处路径故障的核心入口。

重要参数 / 形式

```
namei -l -- PATH
```

长格式显示类型、模式摘要、所有者与组，并跟随符号链接。

```
namei -x -- PATH
```

标记挂载点或文件系统边界；具体字符表现以目标 util-linux 版本为准。

```
namei -n -- PATH
```

不跟随符号链接，用于只观察表面路径分量。

证据边界

它能定位路径层次，但传统权限的完整计算留给第 08 章。

[知识专题] 单根目录树与 FHS：先知道路径大致属于哪里

Linux 把普通文件、设备节点、运行时接口和挂载进来的其他文件系统组织到同一棵目录树中。FHS 目录用途能帮助管理员快速判断“应去哪里找”，但它不替代查询：发行版可能使用兼容符号链接，某个目录也可能是单独挂载点或虚拟文件系统。最稳妥的切入方式是先理解目录职责，再用 `ls -ld`、`readlink`、`findmnt` 等证据确认当前系统的真实布局。

① [知识点] / 是路径解析的根，不等于某一块磁盘

绝对路径以 `/` 开头，并从当前进程看到的根目录开始解析。所有普通路径都接入这棵树，但树中的不同子目录可以来自不同文件系统。因而“同一棵目录树”不等于“所有名称都在同一文件系统”，这也是硬链接可能在两个看似相邻的目录之间失败的原因。

本章只建立这一接口：路径所在文件系统可用 `stat`、`df` 或 `findmnt` 查询。分区、挂载配置、挂载遮蔽和 `/etc/fstab` 留到“文件系统挂载与持久性”章节。

② [知识点] 常用目录按职责理解，而不是孤立背诵

配置与身份

`/etc` 保存主机特定的系统与服务配置；`/home` 是普通用户主目录的常见上级；`/root` 是 `root` 自己的主目录。用户真实主目录仍以账户记录为准。

软件与共享内容

`/usr` 主要承载发行版提供的程序、库和共享数据；`/usr/local` 用于把本地管理员安装内容与发行版管理内容分开；`/boot` 保存启动相关内容。

可变与临时状态

`/var` 保存日志、队列、缓存和数据库等可变持久数据；`/run` 保存当前启动周期的运行状态；`/tmp` 可能被自动清理，不能当作可靠持久存储。

内核与设备接口

`/dev` 提供设备节点；`/proc` 提供进程与内核运行信息；`/sys` 暴露设备、驱动和内核对象。它们都在目录树中，但多数内容不是普通持久磁盘数据。

③ [知识点] RHEL 9 的兼容目录可能本身就是符号链接

现代 RHEL 系统通常把 `/bin`、`/sbin`、`/lib`、`/lib64` 统一到 `/usr` 对应位置，并保留兼容符号链接。学习时不要只背“`/bin` 放普通命令”，而应看到对象关系：表面路径可能是符号链接，最终程序位于 `/usr/bin`。

推荐在目标系统查询：

```
ls -ld /bin /sbin /lib /lib64
readlink /bin
realpath -e /bin
```

这组命令分别显示名称自身、链接保存的文本和规范化后的最终路径。具体布局以目标 RHEL 9 系统的证据为准。

[Cheatsheet] `/` 是统一命名空间的根；`/etc` 放配置，`/var` 放可变持久数据，`/run` 放当前启动周期状态；`/proc`、`/sys`、`/dev` 是运行接口；常见兼容目录先用 `ls -ld` 和 `readlink` 查询。

[知识专题] 路径解析：路径字符串怎样一步步到达对象

路径不是对象身份，而是查找对象的指令。诊断路径问题时，第一步不是立即查看最终文件，而是确定解析起点、拆分路径分量，并找出第一个无法继续的分量。只要把路径解析看成逐层查表，`No such file or directory`、`Not a directory` 和符号链接循环就不再是模糊报错。

① [知识点] 绝对路径和相对路径只是在起点上不同

绝对路径以 `/` 开头，从根目录解析：

```
/etc/ssh/sshd_config
```

相对路径不以 `/` 开头，从当前工作目录解析：

```
configs/app.yml  
../archive/report.txt
```

相对路径的含义依赖当前工作目录。执行脚本或计划任务时，不能假定其当前目录与交互式终端一致；稳定任务应明确工作目录或使用绝对路径。

`pwd` 显示 Shell 当前工作目录：

```
pwd
```

波浪号 `~` 是 Shell 在执行命令前进行的展开，不是内核路径解析中的特殊路径分量。其完整语义属于“Shell 解析、引用与展开”章节。

② [知识点] `.`、`..`、尾随斜线和路径分量都有操作语义

- `.` 表示当前目录；
- `..` 表示父目录；在根目录中继续使用 `..` 仍停留在根；
- `/` 分隔路径分量，不能作为普通文件名的一部分；
- NUL 字符不能出现在路径名中；
- Linux 文件名区分大小写；
- 路径末尾的 `/` 要求解析结果具有目录语义。

例如，对一个普通文件使用 `report.txt/`，可能得到“不是目录”的错误。这个尾随斜线不是无意义装饰，它改变了最终分量的要求。

③ [知识点] 符号链接会把目标文本插入剩余路径继续解析

假设：

```
/srv/app/current -> releases/v2
```

访问：

```
/srv/app/current/bin/start
```

解析到 `current` 时，内核读取链接中的 `releases/v2`，从链接所在目录 `/srv/app` 继续解释相对目标，然后再拼接剩余的 `bin/start`。所以相对符号链接的目标不是相对于“创建链接时的当前目录”，而是相对于“符号链接所在目录”。

④ [知识点] 逻辑路径和物理路径可以同时成立

Shell 可能记录用户通过符号链接进入目录的逻辑路径，而物理路径是展开符号链接后的真实目录链：

```
pwd          # 通常显示逻辑路径
pwd -P       # 展开符号链接，显示物理路径
realpath -e .
```

逻辑路径适合保持用户导航语境，物理路径适合调查对象落点。两者回答不同问题，不应简单判定其中一个“错误”。

[Cheatsheet] 绝对路径从 `/` 开始，相对路径从 `pwd` 开始；`.` 是当前目录，`..` 是父目录；尾随 `/` 要求目录语义；相对符号链接从链接所在目录解释；物理路径用 `pwd -P` 或 `realpath -e`。

[知识专题] 目录项与 inode：名称不是对象本身

人通常通过名称识别文件，内核则需要把名称解析到 inode。这个区别是理解链接和删除行为的核心。看到两个路径内容相同，不代表它们是同一个对象；看到 inode 数字相同，也必须先确认它们属于同一文件系统。

01 路径文本 `/srv/app/current/config.yml`

→

02 逐级目录项名称 → inode

→

03 inode 类型、元数据、链接计数

→

04 对象数据内容或特殊对象接口

判断边界：路径是查找指令，不是对象身份；名称相同、内容相同或 inode 数字相同，都不足以脱离文件系统上下文单独证明“同一对象”。

① [知识点] 目录项保存“名称到 inode”的映射

目录不是“装着文件内容的盒子”；从命名角度看，它维护一组目录项。每个目录项至少把一个名称映射到所属文件系统上的 inode。路径解析本质上是在每一级目录中查找下一个名称。

因此，重命名或在同一文件系统中移动文件，通常主要改变目录项：旧目录删除一条名称映射，新目录增加一条名称映射，对象 inode 可以保持不变。跨文件系统移动无法直接沿用原 inode，工具通常需要复制数据并删除旧名称。

② [知识点] inode 保存对象元数据和数据位置

inode 通常包含：

- 对象类型与模式位；
- 所有者 UID 和组 GID；
- 文件大小和占用块信息；
- 访问、修改、状态变更等时间；
- 硬链接计数；
- 指向文件数据或文件系统数据结构的引用。

文件名不保存在普通文件的 inode 中。一个 inode 可以被多个目录项引用，这正是硬链接的基础。权限的计算规则属于第 08 章，本章只把模式位视为 inode 元数据之一。

③ [知识点] 对象身份至少要比“文件系统 + inode”

inode 编号只需在所属文件系统内唯一。两个路径的 inode 数字碰巧相同，如果设备或文件系统不同，仍然是不同对象。静态调查可使用：

```
stat -c 'device=%d inode=%i links=%h type=%F name=%N' PATH
```

比较两个路径是否为同一对象时，至少核对 %d 与 %i。sha256sum 相同只能证明读取到的字节内容相同，不能证明 inode、所有者、权限、时间或链接关系相同。

④ [知识点] 链接计数与打开引用共同决定对象生命周期

每增加一个普通文件硬链接，inode 的链接计数增加。删除一个名称只是移除一个目录项并减少链接计数。最后一个硬链接被删除后：

- 如果没有进程打开该对象，文件系统可以回收 inode 和数据空间；
- 如果仍有进程持有打开文件描述符，对象会继续存在，直到最后一个引用关闭。

所以“目录中已经看不到文件名”不等于“数据已经立刻从系统中消失”。这也是日志文件被删除后空间可能暂时没有释放的底层原因。

[Cheatsheet] 目录项是“名称 -> inode”；inode 不保存普通文件名；对象身份比较 %d:%i；内容哈希不能证明链接关系；删除名称后还要看剩余硬链接和打开引用。

[知识专题] 文件类型：名称、inode 类型和内容格式是三个维度

Linux 把目录、符号链接、设备节点、FIFO 和 socket 都放在文件系统命名空间中，但它们的操作语义不同。扩展名只是名称的一部分，不决定 inode 类型；file 输出的“文本”“ELF”或“压缩数据”又是内容格式判断。稳定调查要明确自己在问哪一种“类型”。

① [知识点] `ls -l` 的首字符表示对象类型

首字符	类型	常见示例
-	普通文件	配置、文本、程序、归档
d	目录	<code>/etc</code> 、 <code>/var/log</code>
l	符号链接	<code>/bin</code> 等兼容路径
b	块设备	磁盘或逻辑块设备节点
c	字符设备	终端、随机数等字符设备节点
p	FIFO/命名管道	进程间数据流接口
s	socket	Unix 域 socket 路径

类型字符之后的权限位属于第 08 章。本章只训练识别类型并选择合适证据。

② [知识点] `ls`、`stat` 和 `file` 回答不同问题

```
ls -ld -- PATH
stat -- PATH
file -- PATH
```

- `ls -ld` 查看名称自身及摘要，`-d` 防止把目录展开成其内容列表；
- `stat` 查看 inode 元数据，默认不解引用最终符号链接；
- `file` 使用 magic、内容特征等判断文件格式，默认通常把符号链接报告为链接对象。

需要检查链接目标时，可显式使用：

```
stat -L -- LINK
file -L -- LINK
```

③ [知识点] 扩展名和内容不能替代对象类型证据

名为 `report.txt` 的对象可能是目录、符号链接或二进制文件；没有扩展名的程序也可以是 ELF 可执行文件或脚本。考试和工作中都应先用对象证据，再用内容证据：

```
对象类型: ls -ld / stat
内容格式: file
字节一致: sha256sum
```

[Cheatsheet] `ls -l` 首字符看 inode 类型；`stat` 看元数据；`file` 看内容格式；扩展名不是可靠类型；查看目录自身用 `ls -ld`，跟随最终链接用显式 `-L`。

[操作专题] 从路径表面深入到真实对象：选择正确的查询接口

调查路径时，最常见的错误是只执行一条 `ls -l` 就下结论。稳定流程应从命名上下文开始，分别检查名称自身、链接原文、解析链、最终对象和对象身份。每条命令都只证明一个层次。

名称层 `ls -ld` 证明目录项表面状态，不证明整条路径可达。

对象层 `stat` 证明 inode 元数据，不证明内容符合业务目标。

链接原文 `readlink` 证明保存的文本，不证明目标存在。

路径链 `namei -l` 定位逐级解析，不替代权限规则分析。

① [操作] 使用 `pwd` 与 `ls -ld` 固定命名上下文

作用对象：当前 Shell 工作目录和指定目录项。

基本形式：

```
pwd
ls -ld -- PATH
```

关键语义：`--` 结束选项解析，避免以 `-` 开头的名称被误当作参数。`-d` 让目录和指向目录的符号链接以名称自身为观察对象，而不是列出目录内容。

验证边界：`ls -ld` 可显示类型字符和链接箭头，但不能单独证明目标链中的每一级都存在，也不能证明两个路径共享 inode。

② [操作] 使用 `stat` 比较链接自身、目标和对象身份

作用对象：最终路径分量的 inode，或在 `-L` 下解引用后的目标 inode。

```
stat -- PATH
stat -L -- PATH
stat -c 'device=%d inode=%i links=%h type=%F size=%s name=%N' -- PATH
```

常用格式字段：

字段	含义
%d	设备标识的十进制值
%i	inode 编号
%h	硬链接计数
%F	文件类型文本
%s	字节大小
%n	输入名称

字段	含义
%N	带引用和链接目标的名称表示

验证边界：`stat` 证明对象元数据，不证明内容符合业务要求。远程或特殊文件系统还可能存在缓存和实现差异。

③ [操作] 使用 `readlink` 读取符号链接保存的原始目标

作用对象：符号链接自身的数据内容。

```
readlink -- LINK
```

输出可能是绝对路径，也可能是相对文本。它只回答“链接保存了什么”，不保证目标存在，也不保证这段文本从链接父目录解释后能到达预期对象。

不要用 `cat LINK` 读取链接原文；普通文件读取通常会跟随链接并读取目标内容。

④ [操作] 使用 `realpath` 获取规范路径

```
realpath -e -- PATH
realpath -m -- PATH
```

- `-e` 要求所有路径分量存在，适合证明当前路径确实可解析；
- `-m` 允许分量不存在，适合规范化计划创建或当前失效的路径文本。

`realpath -e` 失败而 `readlink` 成功，通常说明链接对象存在，但目标解析链中有缺失或错误分量。

⑤ [操作] 使用 `namei` 查看逐层路径链

```
namei -l -- PATH
namei -x -- PATH
```

`namei` 逐层显示路径分量，遇到符号链接时继续展开，并通过缩进表示上下文。`-l` 以长格式显示类型和模式摘要；`-x` 可标记挂载点或文件系统边界，具体输出以目标系统版本为准。

帮助入口：

```
man stat
man readlink
man realpath
man namei
```

[Cheatsheet] 当前目录 `pwd`；名称自身 `ls -ld`；inode 和链接数 `stat`；链接原文 `readlink`；存在的规范路径 `realpath -e`；逐层链路 `namei -l`。

[知识专题] 硬链接：多个平等名称共享同一个 inode

硬链接不是“指向原文件的快捷方式”，而是同一 inode 的另一个目录项。创建后，两个名称在对象层面地位平等，无法从 inode 判断哪个是“原件”。这种语义决定了硬链接的验证方式、文件系统边界和删除行为。

① [知识点] 创建硬链接只增加名称和链接计数

```
ln -- TARGET NEW_NAME
```

成功后，`TARGET` 与 `NEW_NAME` 应在同一文件系统中具有相同 inode，链接计数相应增加：

```
stat -c '%d:%i links=%h %N' -- TARGET NEW_NAME
```

通过任一名称修改文件内容，另一个名称读取到同一对象的数据。所有者、权限和普通文件时间属于共享 inode，不存在为每个硬链接单独保存一套普通文件元数据。

② [知识点] 硬链接不能跨文件系统

目录项引用的是所属文件系统内部的 inode。新名称若位于另一个文件系统，就不能直接引用源文件系统的 inode，典型失败表现为 `Invalid cross-device link`。

调查时先比较目标和新名称父目录：

```
stat -c 'device=%d mount=%m name=%N' -- TARGET NEW_PARENT
```

需要跨文件系统保持引用关系时，通常选择符号链接；需要独立数据时选择复制。二者终态不同，不能互换声称完成。

③ [知识点] 普通工作流不为目录创建硬链接

目录硬链接会让目录图出现额外父路径和潜在循环，破坏树状遍历和生命周期管理。Linux 文件系统保留 `.`、`..` 等受控目录关系，但普通管理员不应尝试用强制参数为目录创建硬链接。题目要求链接目录时，使用符号链接，并明确验证解析结果。

④ [知识点] 删除任意一个硬链接只删除一个名称

```
rm -- ONE_NAME
```

只要仍有其他目录项引用 inode，数据就能通过剩余名称访问。最后一个硬链接删除后，还要看是否存在打开文件描述符。硬链接的稳态判断应基于链接计数和可访问名称，不要把某个名称称为永久“源文件”。

[Cheatsheet] `ln TARGET NAME` 创建同 inode 的新名称；用 `%d:%i` 验证；链接计数增加；不能跨文件系统；不为目录创建硬链接；删除一个名称不会影响剩余硬链接。

[知识专题] 符号链接：独立对象保存一段目标路径

符号链接与硬链接最根本的区别不是命令多了 `-s`，而是对象模型不同。符号链接拥有自己的 inode，文件内容是一段路径文本；访问者在允许跟随链接的场景中，再解析这段文本到目标对象。因此它可以跨文件系统、可以指向目录、可以先于目标创建，也会发生悬空和重新绑定。

① [知识点] 链接自身、目标文本和最终对象必须分开观察

```
ln -s -- TARGET_TEXT LINK_NAME
```

创建后有三层事实：

1. `LINK_NAME` 是一个符号链接对象，有自己的 inode；
2. `readlink LINK_NAME` 返回保存的 `TARGET_TEXT`；
3. 跟随该文本后可能到达一个目标对象，也可能失败。

对应证据：

```
stat -- LINK_NAME
readlink -- LINK_NAME
stat -L -- LINK_NAME
```

② [知识点] 绝对目标和相对目标具有不同迁移特性

绝对目标以 `/` 开头，从当前进程根目录解析：

```
ln -s /srv/app/releases/v2 /srv/app/current
```

相对目标从链接所在目录解析：

```
ln -s releases/v2 /srv/app/current
```

当整个 `/srv/app` 目录树一起迁移时，相对链接通常更便携；只移动链接本身而不保持相对布局，则可能失效。选择哪一种取决于目标终态，不是“相对永远更好”或“绝对永远更清楚”。

③ [知识点] 符号链接可以跨文件系统并指向目录

符号链接只保存文本，不直接引用目标 inode，所以链接对象和目标可以位于不同文件系统，也可以把目录作为目标。跨文件系统只是允许创建，不代表目标一定存在或可访问。

④ [知识点] 悬空链接仍然是存在的对象

目标删除或移动后，符号链接本身仍存在：


```
ls -ld -- LINK_NAME
readlink -- LINK_NAME
```

但跟随解析会失败：

```
realpath -e -- LINK_NAME
stat -L -- LINK_NAME
```

如果以后在同一路径创建一个新对象，符号链接会解析到新对象，而不是继续绑定旧 inode。这种“按名称重新绑定”是符号链接和硬链接生命周期差异的重要考点。

⑤ [边界] 删除符号链接时不要给目录链接追加 `/`

要删除指向目录的符号链接本身，应对链接名称执行：

```
rm -- LINK_NAME
```

不要使用 `rm -r LINK_NAME/` 作为默认操作。尾随 `/` 会要求目录语义并可能让工具进入目标目录上下文，扩大误操作风险。删除前先用 `ls -ld` 和 `test -L` 确认对象。

[Cheatsheet] 符号链接有自己的 inode；原始文本用 `readlink`；目标用 `stat -L`；相对目标从链接所在目录解释；可跨文件系统、可指向目录、可悬空；删除链接本身不要追加 `/`。

[操作专题] 创建、验证、替换和删除链接的完整闭环

链接操作看似只有一条 `ln`，真正容易失分的是目标顺序、已有名称、目录目标语义和验证不足。稳定做法是操作前建立基线，创建后分别验证名称、链接原文、对象身份和最终解析结果；替换时只删除已经确认的链接对象。

① [操作] 创建并验证硬链接

作用对象：已存在普通文件的 inode 与新目录项。

```
ln -- /srv/linklab/source/report.txt \
    /srv/linklab/archive/report.hard
```

验证：

```
ls -li -- /srv/linklab/source/report.txt \
    /srv/linklab/archive/report.hard
stat -c 'device=%d inode=%i links=%h type=%F name=%N' -- \
    /srv/linklab/source/report.txt \
    /srv/linklab/archive/report.hard
```

验收重点是相同的设备标识和 inode，不是名称相似或内容哈希相同。

② [操作] 创建并验证符号链接

```
ln -s -- releases/v2 /srv/reporting/current
```

分层验证：

```
ls -ld -- /srv/reporting/current
readlink -- /srv/reporting/current
namei -l -- /srv/reporting/current/bin/report
realpath -e -- /srv/reporting/current/bin/report
stat -- /srv/reporting/current
stat -L -- /srv/reporting/current
```

如果目的名称已存在为目录，`ln` 可能把新链接创建到该目录内。需要把目的名称严格当作一个路径时，可在确认目标系统支持后使用 `-T / --no-target-directory`，或先调查并清理明确的旧名称。

③ [操作] 安全替换现有符号链接

不要直接把 `ln -sf` 当作默认答案。先调查：

```
ls -ld -- /srv/reporting/current
if test -L /srv/reporting/current; then
    readlink -- /srv/reporting/current
fi
```

确认它确实是允许替换的符号链接后，执行最小修改：

```
rm -- /srv/reporting/current
ln -s -- releases/v2 /srv/reporting/current
```

随后重新执行原始验证链。真实业务环境还应评估并发访问和原子切换需求；RHCSA 静态任务至少不能在未调查时删除未知目录或普通文件。

④ [操作] 删除名称并验证生命周期

删除一个硬链接名称：

```
rm -- /srv/linklab/source/report.txt
stat -c '%d:%i links=%h %N' -- /srv/linklab/archive/report.hard
```

删除符号链接名称：

```
rm -- /mnt/labfs/report.soft
```

`unlink NAME` 也能删除单个非目录名称，但考试和日常工作中 `rm -- NAME` 更常见。无论使用哪个入口，都要理解它改变的是目录项。

[Cheatsheet] 操作前 `ls -ld`；硬链接 `ln TARGET NAME`，验证 `%d:%i`；符号链接 `ln -s TARGET_TEXT NAME`，验证 `readlink + namei + realpath`；替换前确认 `test -L`；删除链接本身不加 `/`。

[诊断专题] 路径与链接故障：从症状推进到第一处失败

路径故障经常在最终命令处只显示一句错误。高质量诊断不是反复重建链接，而是先保留当前状态，再找出第一处无法解析的分量。统一链路如下：

```
症状
-> 当前目录与输入路径
-> 名称自身
-> 链接原文
-> 逐层路径链
-> 最终对象或文件系统身份
-> 最小修复
-> 原链路再验证
```

① [诊断点] `readlink` 有输出，但 `realpath -e` 失败

症状：链接看起来存在，访问目标却报告不存在。

当前证据：

```
ls -ld -- LINK
readlink -- LINK
```

假设：链接对象存在，但目标文本从链接父目录解析后包含缺失分量、错误层级或已经移动的名称。

下一条最有区分度的证据：

```
namei -l -- LINK
realpath -e -- LINK
```

最小修复：只修改错误的链接名称或目标文本，不修改无关目录和权限。

② [诊断点] `Not a directory` 表示中间分量类型错误

例如 `/srv/app/config/main.yml` 中，`config` 实际是普通文件或指向普通文件的链接。最终 `main.yml` 是否存在已经不是首要问题。

```
namei -l -- /srv/app/config/main.yml
```

找到第一个不是目录却有剩余分量的对象。若问题是权限导致无法穿越，记录证据并转到“本地权限、umask 与特殊权限”章节，不在本章用 `chmod 777` 绕过。

③ [诊断点] Too many levels of symbolic links 优先调查链接环

常见结构：

```
A -> B
B -> A
```

或更长的间接循环。保留两个链接对象，使用：

```
ls -ld -- A B
readlink -- A
readlink -- B
namei -l -- A
```

最小修复是纠正其中一个目标文本，使路径链收敛到实际对象；不要无差别删除整组目录。

④ [诊断点] Invalid cross-device link 说明硬链接跨越文件系统边界

```
stat -c 'device=%d mount=%m name=%N' -- SOURCE DEST_PARENT
```

如果设备标识或挂载点不同，硬链接的对象模型不成立。根据任务选择：

- 需要按路径引用同一目标：符号链接；
- 需要独立副本：复制；
- 题目必须硬链接：把新名称放到源文件所在文件系统。

不能用符号链接替代硬链接后仍声称满足“同 inode”验收。

⑤ [诊断点] 名称已删除但空间或进程访问仍存在

假设链：

```
仍有其他硬链接
-> 链接计数尚未归零
-> 或某进程仍持有打开文件描述符
```

先检查已知剩余名称的 `stat`。若链接计数已经归零但已识别进程仍在使用对象，可在进程章节的帮助下检查 `/proc/<PID>/fd/`，或在系统安装 `lsuf` 时使用相应查询。不要为了释放空间直接无调查终止进程。

⑥ [边界] 最终符号链接是否跟随取决于具体命令

路径中间分量的符号链接通常需要展开才能继续解析；最终分量是否跟随则可能受命令和参数影响。例如：

- `stat LINK` 检查链接自身；
- `stat -L LINK` 检查目标；
- `readlink LINK` 读取链接文本；
- `rm LINK` 删除链接名称。

诊断时必须说明使用的是哪一种语义，不能把所有命令都概括为“默认跟随链接”。

[Cheatsheet] 悬空链接：`readlink -> namei -l -> realpath -e`；非目录分量：找第一处类型错误；链接环：保留并展开链；跨文件系统：比较设备与挂载点；删除后仍占用：查其他硬链接和打开引用。

[经典任务] 调查并修复失效的相对符号链接

环境与当前状态

应用目录如下：

```
/srv/reporting/releases/v2/bin/report
/srv/reporting/current
```

`/srv/reporting/releases/v2/bin/report` 是题目指定的正确对象。应用固定通过以下路径访问：

```
/srv/reporting/current/bin/report
```

当前 `current` 是一个符号链接，但该访问路径失败。你不知道链接保存的文本是否正确。

目标终态

- `/srv/reporting/current` 必须是符号链接；
- 链接保存的目标必须是相对文本 `releases/v2`；
- `/srv/reporting/current/bin/report` 必须解析到题目给定对象；
- 不修改 `releases/v2` 中任何文件内容；
- 不修改权限、所有者、SELinux 或挂载配置；
- 不使用无调查的 `ln -sf`；
- 保留调查和验收命令。

验收证据

```
名称层: ls -ld
原始链接层: readlink
路径链层: namei -l
规范路径层: realpath -e
对象层: stat / stat -L
内容类型层: file
```

请先独立完成。参考解答从新页开始。

[经典任务] 建立硬链接与符号链接，并观察删除名称后的分化

环境与当前状态

题目已经准备：

```
/srv/linklab/source/report.txt
/srv/linklab/archive/
/mnt/labfs/
```

题目同时保证 `/mnt/labfs` 与 `/srv/linklab` 属于不同文件系统。本章不负责创建或挂载文件系统。

目标终态与操作过程

1. 在 `/srv/linklab/archive/` 创建 `report.hard`，它必须是源文件的硬链接；
2. 在 `/mnt/labfs/` 创建 `report.soft`，其目标文本必须是绝对路径 `/srv/linklab/source/report.txt`；
3. 证明硬链接与源名称共享对象；
4. 证明符号链接拥有独立 inode，但跟随后到达源对象；
5. 删除原名称 `/srv/linklab/source/report.txt`；
6. 证明硬链接仍访问旧对象，而符号链接已经悬空；
7. 在原路径创建一个新的普通文件；
8. 证明符号链接现在解析到新对象，而硬链接仍引用旧对象。

限制条件

- 不得把复制品冒充硬链接；
- 不得在 `/mnt/labfs` 尝试创建跨文件系统硬链接后忽略失败；
- 不得删除 `archive/report.hard`；
- 所有判断必须用设备标识、inode、链接计数和链接原文证明；

- 不依赖伪造的固定 inode 数字或命令输出。

[参考解答] 任务一：按路径链调查并最小修复

① 建立名称与链接原文基线

```
ls -ld -- /srv/reporting/current
readlink -- /srv/reporting/current
```

先确认 `current` 确实是符号链接，并记录它当前保存的文本。若它是普通文件或目录，不能直接删除，应停止并重新确认题目环境。

② 展开当前失败链

```
namei -l -- /srv/reporting/current/bin/report
realpath -e -- /srv/reporting/current/bin/report
```

`namei -l` 用于定位第一处错误分量。`realpath -e` 失败只说明当前完整路径不能全部解析；不能据此直接修改权限或创建未知目录。

③ 核对题目指定目标

```
ls -ld -- /srv/reporting/releases/v2 \
/srv/reporting/releases/v2/bin/report
stat -c 'device=%d inode=%i links=%h type=%F name=%N' -- \
/srv/reporting/releases/v2/bin/report
file -- /srv/reporting/releases/v2/bin/report
```

这一步证明目标名称存在及其对象类型，不声称已经执行或通过业务功能测试。

④ 只替换已经确认的符号链接

```
if test -L /srv/reporting/current; then
    rm -- /srv/reporting/current
    ln -s -- releases/v2 /srv/reporting/current
else
    printf '%s\n' 'current 不是符号链接，停止修改' >&2
fi
```

相对文本 `releases/v2` 从 `/srv/reporting` 解释，得到 `/srv/reporting/releases/v2`。

⑤ 分层再验证

```
ls -ld -- /srv/reporting/current
readlink -- /srv/reporting/current
namei -l -- /srv/reporting/current/bin/report
realpath -e -- /srv/reporting/current/bin/report
stat -- /srv/reporting/current
stat -L -- /srv/reporting/current/bin/report
file -- /srv/reporting/current/bin/report
```

验收时应明确：

- `ls -ld` 证明 `current` 是链接；
- `readlink` 证明原始文本为 `releases/v2`；
- `realpath -e` 证明完整路径当前可解析到 `v2`；
- `stat` 与 `stat -L` 分别观察链接自身和目标对象；
- `file` 只判断内容类型，不代替业务执行验证。

典型错误

- 在 `/srv/reporting` 中运行 `ln -s ../releases/v2 current`：该文本会从链接父目录解释为 `/srv/releases/v2`；
- 使用 `ln -sf` 覆盖未知对象：可能隐藏原有目录或错误名称状态；
- 只看 `readlink` 输出就声称修复：目标文本存在不等于完整链可解析；
- 修改权限解决 `ENOENT`：错误层级尚未确认。

[参考解答] 任务二：用对象身份观察硬链接与符号链接分化

① 确认文件系统边界与当前对象

```
stat -c 'device=%d mount=%m type=%F name=%N' -- \
/srv/linklab/source/report.txt \
/srv/linklab/archive \
/mnt/labfs
```

确认源文件与 `archive` 在同一文件系统，而 `/mnt/labfs` 不同。题目已经给出该条件，命令用于保留验收证据。

② 创建硬链接和符号链接

```
ln -- /srv/linklab/source/report.txt \  
    /srv/linklab/archive/report.hard  
  
ln -s -- /srv/linklab/source/report.txt \  
    /mnt/labfs/report.soft
```

③ 删除前建立三对象基线

```
stat -c 'device=%d inode=%i links=%h type=%F name=%N' -- \  
    /srv/linklab/source/report.txt \  
    /srv/linklab/archive/report.hard \  
    /mnt/labfs/report.soft  
  
readlink -- /mnt/labfs/report.soft  
stat -L -c 'device=%d inode=%i links=%h type=%F name=%N' -- \  
    /mnt/labfs/report.soft
```

应从关系上判断：

- 源名称与 `report.hard` 的设备标识和 inode 相同；
- `report.soft` 自身 inode 不同；
- `stat -L report.soft` 到达的设备与 inode 与源对象相同；
- `readlink` 显示绝对目标文本。

④ 删除原名称并验证分化

```
rm -- /srv/linklab/source/report.txt  
  
stat -c 'device=%d inode=%i links=%h type=%F name=%N' -- \  
    /srv/linklab/archive/report.hard  
  
ls -ld -- /mnt/labfs/report.soft  
readlink -- /mnt/labfs/report.soft  
realpath -e -- /mnt/labfs/report.soft
```

最后一条命令此时应无法证明存在的最终路径。不要伪造具体错误文本；不同语言环境的提示可能不同。

⑤ 在原路径创建新对象并再次比较

```
printf '%s\n' 'new generation' > /srv/linklab/source/report.txt

stat -c 'device=%d inode=%i links=%h type=%F name=%N' -- \
    /srv/linklab/source/report.txt \
    /srv/linklab/archive/report.hard

stat -L -c 'device=%d inode=%i links=%h type=%F name=%N' -- \
    /mnt/labfs/report.soft
realpath -e -- /mnt/labfs/report.soft
```

最终关系应为：

```
archive/report.hard -> 仍引用删除前的旧 inode
source/report.txt   -> 新创建的 inode
report.soft         -> 按目标路径解析到新 inode
```

⑥ 内容层辅助验证

在对象身份已经证明后，可以用内容作为辅助证据：

```
cat -- /srv/linklab/archive/report.hard
cat -- /srv/linklab/source/report.txt
cat -- /mnt/labfs/report.soft
```

即使内容偶然相同，也不能替代 `%d:%i` 的身份判断。

典型错误

- 只比较 inode 数字，不比较文件系统；
- 在 `/mnt/labfs` 创建硬链接失败后改用复制，却仍声称“同一对象”；
- 删除原名称后把悬空符号链接当作已经删除；
- 原路径重建后认为硬链接也自动指向新对象；
- 只用 `sha256sum` 判断链接类型和 inode 关系。

[本章收束] 从“看见一个路径”升级为“识别一条对象解析链”

本章最重要的迁移不是多记几条命令，而是改变判断顺序：

路径文本

- > 解析起点
- > 每一级目录项
- > inode 和文件类型
- > 是否经过符号链接
- > 最终对象身份
- > 名称删除后的剩余引用

遇到文件问题时，先问自己正在证明哪一层：

- `ls -ld` 证明名称自身；
- `stat` 证明 inode 元数据；
- `file` 证明内容格式；
- `readlink` 证明链接原文；
- `realpath` 证明规范路径；
- `namei` 证明逐层解析链；
- `%d:%i` 证明对象身份；
- 链接计数和打开引用解释生命周期。

下一章将在这个对象模型上学习文件查找、文本筛选与批量处理。届时 `find` 选择的不是抽象“文件名”，而是目录树中满足类型、元数据和路径条件的一组对象。

主要判断表

你要回答的问题	首选证据	它不能单独证明
当前 Shell 从哪里解释相对路径？	<code>pwd / pwd -P</code>	目标是否存在
名称自身是什么类型？	<code>ls -ld -- PATH</code>	符号链接目标链是否完整
两个名称是否共享对象？	<code>stat -c '%d:%i %h'</code>	内容是否满足业务要求
链接保存了什么文本？	<code>readlink -- LINK</code>	文本能否解析成功
路径最终规范化到哪里？	<code>realpath -e -- PATH</code>	原始链接文本是什么
哪一级目录开始失败？	<code>namei -l -- PATH</code>	失败是否由第 08 章权限规则造成
删除名称后对象是否仍存活？	链接计数 + 打开引用	哪个业务进程应被终止

向下一章交接

下一章“文件查找、文本筛选与批量处理”会把本章对象模型变成选择条件：`find` 遍历的是目录项和对象状态，类型、路径、时间和所有者都只是筛选维度。进入下一章前应保持一个习惯：先确认自己正在选择名称、路径还是对象状态，再决定是否批量操作。

最终 Cheatsheet

```
pwd
ls -ld -- PATH
stat -c 'device=%d inode=%i links=%h type=%F name=%N' -- PATH
stat -L -- LINK
file -- PATH
readlink -- LINK
realpath -e -- PATH
namei -l -- PATH
ln -- TARGET HARD_LINK
ln -s -- TARGET_TEXT SYMLINK
rm -- NAME
```

硬链接：同文件系统、同 inode、平等名称、删除一个名称仍可存活

符号链接：独立 inode、保存路径文本、可跨文件系统、可悬空和重新绑定

删除：先删除目录项；最后硬链接和最后打开引用都消失后，对象才可最终回收